

# StateWake

## Evidence-Backed Reliability Infrastructure for Stateful AI Systems

---

Aditya Gupta

21 September 2026 · StateWake v0.4.0 · Apache-2.0

**A reliability claim should be explainable as a verified relationship among evidence, provenance, state transition, recovery history, and decision—not merely inferred from a successful final output or a high evaluation score.**

### ABSTRACT

Stateful AI workflows combine models, tools, retrieved information, external services, operational policies, and long-running execution. Their final outputs can conceal missing evidence, inconsistent state, recovery after failure, or changes in the material on which a decision depended. I developed StateWake as a framework-neutral reliability-evidence layer that makes those conditions explicit and independently checkable. Its central primitive, `ReliabilityEvidenceChain`, binds authoritative evidence references to provenance, integrity, verification, reliability state, reconciliation/recovery, and decision context. Version 0.2.0 added bounded cybersecurity and assurance capabilities; version 0.3.0 added a durable workspace and dataset architecture. Version 0.4.0 makes AI workflow evidence first-class through typed contracts, claim profiles, human-readable verification reports, restart-safe workspace operations, producer integrations, release-trust records, and a reproducible comparative fixture study. The result is not a claim to make arbitrary AI reasoning correct. It is an engineering mechanism for stating **what reliability is being claimed, on which evidence, under which verification and trust conditions, at which point in a changing history**. This paper explains the implemented v0.4.0 architecture, its boundaries, its relationship to established work, and the evidence currently available to evaluate its practical value.

**Keywords:** AI reliability; evidence provenance; verifiable state; security assurance; durable datasets; independent verification.

# 1. The engineering problem

An AI application may complete a request yet leave its operators unable to answer four consequential questions: What exactly supported this outcome? Was the supporting material intact and attributable to its purported source? Which state was justified before and after a discrepancy or recovery? Could another party verify the claim without trusting an opaque success label? Traces, evaluation results, provenance records, and checkpoints each address part of that problem. They are valuable inputs; they are not automatically the same thing as a durable, verified reliability claim. OpenTelemetry describes execution signals, and OpenLineage models the provenance of jobs, runs, and datasets [5], [6]. Agent-oriented research has extended provenance to prompts, decisions, and cross-system interactions [11], [12].

Consider a workflow that analyzes a document using an external tool. The tool times out, a retry succeeds, and the system returns *accepted*. A later revision changes the document. A final status and trace may show activity, but the reliability question is more precise: **which document version was accepted, which failed attempt occurred, which recovery explains the subsequent result, and what remains valid after the revision?** Reconstructing that answer should not depend on undocumented joins across multiple products or on treating later success as if the initial failure never happened.

The design objective is therefore limited but useful: record evidence-linked reliability states; enforce declared transition and verification conditions; preserve discrepancy and recovery relationships; allow bounded operational decisions; and make the history durable and usable after the original execution finishes.

# 2. Architectural contribution and boundary

I use *evidence* to mean an identifiable artifact or record supplied by a producer, *integrity* to mean checks on bytes and relationships, *provenance* to mean explicit derivation/source relationships, *reliability state* to mean a declared state of a subject under defined conditions, and *attestation* to mean a claim bound to verified material. Integrity does not prove that a statement is true or that a model reasoned correctly. A signature identifies a signing relationship only under an accepted trust policy. A successfully recovered run does not erase the prior failure.

StateWake's composition is:



**Fig. 1.** Conceptual flow of StateWake v0.4.0. Actual integration can revisit verification and state as evidence changes; the figure is not a claim that every input follows an identical linear execution path.

The central `ReliabilityEvidenceChain` composes references to a run, state, evidence, provenance, and integrity record, with optional comparison, reconciliation, recovery, attestation, and decision-basis references. It deliberately does **not** become a second authoritative copy of every source-system payload. The product sits beside agent runtimes, evaluation systems, observability, storage, and CI/CD; it is not an agent orchestrator, hosted control plane, general test framework, or distributed task queue.

## 3. Evidence, provenance, verification, and reliability state

### 3.1 Admission is a trust boundary

StateWake can ingest bytes and files using producer identity/type, source reference, optional producer version, event/run identity, capture time, and metadata. Receipts bind a producer occurrence to the artifact digest and provide a stable basis for indexing. Native sources and external sources have distinct admission boundaries. Available adapters cover files, CI/CD artifacts, agent-run logs, evaluation output, incident/recovery records, and OpenTelemetry traces. An adapter translates evidence; it does not automatically certify that the upstream producer was honest.

A content-addressed artifact store and receipt store preserve the canonical evidence objects. Evidence references carry kind, identity, digest, optional source, and optional receipt binding. Duplicate identity with inconsistent content is an error rather than an invitation to silently reinterpret history. Source reference and captured time are not interchangeable with semantic truth.

### 3.2 Provenance and deterministic integrity

The provenance model includes nodes with explicit identities, digests, and `derived_from` edges. Verification can reject unresolved edges, self-reference/cycles, and digest mismatches within the declared proof boundary. A proof can record graph integrity, required-edge checks, identity checks, and reachability. StateWake's approach is informed by the distinction between provenance relationships and their validity in W3C PROV and its constraints [1], [2]; **it does not claim full W3C PROV conformance or interchange compatibility merely because the ideas are related.**

The verification result is bounded: it can show that certain bytes and declared relationships are consistent with a proof and trust context. It cannot conclude that arbitrary retrieved statements, model inferences, or real-world decisions are substantively correct.

### 3.3 Reliability is a versioned claim, not a scalar score

The implemented reliability-state vocabulary includes `unknown`, `reliable`, `degraded`, `unreliable`, and `recovered`. A `ReliabilityStateTransition` binds subject, prior/new state, actor, time, evidence-chain identity/digest, decision, rationale, and previous-transition digest. The allowed-transition model is explicit, rather than accepting arbitrary labels as state changes. A later transition does not retroactively mutate an earlier one.

For the evidence chain itself, acceptance is constrained: verification must be `verified`, reconciliation must be `verified` or `recovered`, and reliability state must be `reliable` or `recovered`. Rejection requires a `degraded` or `unreliable` state. These are **implemented local state/decision invariants**, not a claim that every host application adopts the same business policy.

An intuitive expression of the boundary is:

```
Declared claim C(s,t)
  = current state s at time t
  + exact supporting evidence identities/digests
  + verification conditions and trust context
  + prior transition and discrepancy/recovery links
  + decision basis
```

This is a conceptual statement of required relationships, not a new probability calculus. StateWake makes those relationships explicit enough for validation and retrospective reconstruction.

### 3.4 Failure and recovery are part of the same history

Behavioral comparisons and outcome-verification services connect a result to the inputs against which it was checked. Reconciliation binding ties a discrepancy to its specific reconciliation/recovery evidence. The recovery service verifies a declared recovery outcome; it does not execute arbitrary host jobs. A failed occurrence may be retried or recovered, but a later success cannot, by itself, establish that the original attempt succeeded. Persisted state and linked evidence distinguish *the failure*, *the recovery operation*, and *the resulting state*.

Attestations, signed envelopes, claim profiles, proof-completeness reports, release-proof construction, and proof-bundle verification provide additional ways to express and independently check the bounded claim. Trust in a signing key and trust in the decision policy remain different judgments.

## 4. The v0.2.0 cybersecurity and assurance expansion

Security is not bolted onto a final status label. The v0.2.0 implementation adds explicit security-domain models and adapters around evidence and reliability operations. These are meaningful capabilities, but a library containing security controls is not equivalent to an automatically secured deployment.

| Assurance domain                           | Implemented mechanism or record boundary                                                         |
|--------------------------------------------|--------------------------------------------------------------------------------------------------|
| Security and continuous assurance          | Verification findings, trust context, security decisions, and explainable exceptions             |
| Recovery and resilience                    | Recovery outcomes, preserved failure evidence, incident/recovery verification                    |
| Trust domains and anchors                  | Signed checkpoints, local-tip/sequence comparison, key and trust status                          |
| Supply chain and release                   | Artifact integrity, release-proof and attestation-oriented workflows                             |
| Portability and independent verification   | Bounded proof bundles, completeness and verification services                                    |
| Identity, privacy, incident, and longevity | Scoped grants, containment checks, retention/disclosure, forensic timeline, key migration models |

### 4.1 Access, containment, and information handling

`AuthorizationPolicy` uses principals, roles, resource scopes, operation grants, resource state, expiry, and revocation with a deny-by-default decision model. Protected operations include read, ingest, verify, transition, reconcile, recover, attest, delete, retain, and trust/key administration. The WSGI security wrapper accepts host-provided authentication, authorization, request-admission, and security-event interfaces. A host still supplies real identities, secrets handling, TLS, isolation, and network controls.

Bounded JSON parsing, input/archive constraints, reference validation, and verification deadlines limit classes of hostile or malformed input; they do not constitute a process sandbox or universal denial-of-service defense. Governance and privacy utilities support sensitivity classification, metadata redaction, disclosure ceilings, retention requirements, deletion records, and legal hold. **Confidentiality depends on deployment choices as well as package-level policy checks.**

### 4.2 Incident evidence, trust, and cryptographic continuity

Security-incident models capture affected trust state, key compromise impact, evidence references, recovery evidence, post-recovery verification, and a reconstructable timeline. Hash-linked audit records and signed trust checkpoints help detect inconsistent histories within the stated trust model. Key lifecycle, key rotation/revocation, historical-signature verification, algorithm identity, and migration records support the *management of trust changes*. They are not a claim that post-quantum signatures or all future cryptographic migrations are implemented.

The security-assurance decision function maps reliability state, verification status, trust status, evidence references, and recovery requirement to bounded outcomes: `ALLOW`, `ALLOW_WITH_LIMITATIONS`, `REQUIRE_REVERIFICATION`, `QUARANTINE`, `REJECT`, `REVOKE_TRUST`, or `RECOVERY_REQUIRED`. Decision records retain policy identity/version, rationale, verification inputs, and residual risk. Operational exceptions have their own explicit authorization and expiry fields and must not rewrite underlying assurance state.

### 4.3 Supply-chain and independent-verification boundary

Artifact hashes, package metadata, provenance-oriented release proof, signing/trust mechanisms, and portable verification address different parts of supply-chain confidence. The engineering pattern is related to in-toto attestations and SLSA provenance [3], [4]; **no SLSA level or independent certification is claimed**. The package security policy explicitly excludes automatic host hardening, caller authentication, tenant isolation, TLS termination, immutable storage, business-policy correctness, and protection against stolen keys. This boundary aligns with the distinction between software controls and organization-wide risk management in NIST guidance [7]–[10].

## 5. The v0.3.0 persistence and dataset architecture

A reliability record that survives only as a one-off artifact is difficult to operate, investigate, compare, or disclose selectively. Version 0.3.0 addresses this by adding a **workspace**: a durable, versioned local boundary for canonical evidence references, an operational index, historical queries, lifecycle policy, and reproducible exports. It extends the same reliability system rather than inventing a parallel truth store.

### 5.1 Three data roles, not one database

| Data role                   | Responsibility                                                                                                     | Authority                                                                   |
|-----------------------------|--------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| Canonical evidence          | Artifact bytes and producer receipts; state/proof objects managed by existing StateWake components                 | Original StateWake stores and referenced producers                          |
| Operational workspace index | Workspace identity, indexed records/runs, relationships, retention/deletion/export metadata, status and timestamps | Local index of those canonical records; not a semantic replacement for them |
| Dataset projection          | Normalized rows, selected relationships, runs, transitions, lifecycle information                                  | Derived view for query, disclosure, analysis, interchange                   |

The packaged `StateWakeWorkspace` creates/reopens a workspace manifest, directories, and a SQLite-backed reference repository. Opening checks the supported schema and repository integrity; an unsupported or corrupt database is not silently rebuilt as an empty history. Workspace identity and public API contract version are recorded. An injected repository protocol and SQLAlchemy integration surface permit backend abstraction, while the included reference workspace uses SQLite.

### 5.2 Durable ingestion and historical queries

Workspace ingestion reuses the established evidence-admission path and indexes the resulting canonical receipt. SQLite transactions register records and detect identity conflicts. Queries support record/run/event/producer identity, verification and reliability status, sensitivity ceiling, captured-time range, and bounded pagination. Normalized projections carry receipt ID, digest and size, producer identity/version, source reference, event/run identity, capture time, sensitivity, policy, verification status, reliability state, and creation time. Dataset projections also represent relationship, run, state-transition, retention, and deletion concepts.

This is a **history-query surface**, not a guarantee that every possible upstream event is automatically captured. The host must connect producers and select the evidence it wants StateWake to manage.

### 5.3 Exports and analytical access

| Format / surface     | v0.3.0 role                                                    | Dependency boundary   |
|----------------------|----------------------------------------------------------------|-----------------------|
| CSV and JSON         | Interchange and bounded structured projections                 | Standard-library path |
| Portable dataset ZIP | Manifest, dataset, receipt/state projections, member checksums | Standard-library path |
| Parquet              | Columnar analytical export                                     | Optional PyArrow      |
| XLSX                 | Human-readable tabular reporting                               | Optional openpyxl     |
| DuckDB               | Read-only analytical queries and snapshots over Parquet        | Optional DuckDB       |
| SQLAlchemy adapter   | Repository integration surface                                 | Optional SQLAlchemy   |

The `workspace` optional extra declares DuckDB, PyArrow, openpyxl, and SQLAlchemy. Export metadata records the query definition, disclosure ceiling, schema version, output digest/path, creation time, and row count where applicable. The portable dataset bundle explicitly marks `proof_bundle: false`: **a dataset ZIP with checksums is not a cryptographically sufficient reliability proof**. Proof bundles belong to the separate reliability verification boundary described above.

### 5.4 Retention, restart, and verification

Lifecycle operations model retention policy, sensitivity, legal hold, deletion records, export retention, and bounded disclosure. Workspace locking and temporary-file recovery support local operational continuity; storage reporting and diagnostics expose workspace conditions. Workspace verification checks manifest identity, index/reference

consistency, receipts, canonical artifacts, exported files, and orphans. Its statuses include healthy, healthy with limitations, incomplete, invalid, and corrupt. Warnings are therefore not automatically equated with clean verification, and an index row does not by itself prove an artifact still exists and matches its digest.

## 6. The v0.4.0 AI-system reliability expansion

Version 0.4.0 adds seven integrated capabilities without replacing the existing ReliabilityEvidenceChain, workspace, security-assurance, or proof authorities. The new modules formalize what an AI producer emits, what evidence a particular claim requires, how a person inspects that claim, how the local history survives a move or restart, and how release claims and comparative questions are recorded. They extend the same evidence lifecycle rather than creating an independent AI evaluation engine.

### 6.1 Typed AI evidence contracts

The `statewake.ai_contracts` family captures eight categories of AI-system facts: **prompt construction, model invocation, tool calls, retrieval, policy decisions, evaluator output, human approval, and runtime traces/retries/recovery**. Contracts carry schema and contract versions, producer and run identities, timezone-aware UTC capture time, and digest-bound serialized material. Canonical JSON serialization makes the captured representation reproducible. Each contract can become an existing `EvidenceItem`; the caller explicitly persists serialized contract bytes through `StateWakeWorkspace.ingest(...)`. The contract records *what the producer reported*, not a judgment that its model was intelligent, its tool was safe, or its source was true.

For a retrieval-augmented answer, the relevant capture includes prompt/model evidence and retrieval source or snapshot identifiers, together with digests of the retrieved material. For a tool action, it can include the tool schema, input/output identity, side-effect classification, and policy evidence. A human approval record preserves the actor reference, role, scope, basis, and time; the presence of that record does not itself decide whether the approval was authorized under an external policy.

### 6.2 Versioned claim profiles and bounded decisions

The new built-in profile catalog expresses *minimum evidence for a named claim*, not a single universal score. Eight versioned profiles are available:

| Built-in claim profile                           | Declared claim boundary                                        |
|--------------------------------------------------|----------------------------------------------------------------|
| <code>rag_answer_verified.v1</code>              | Prompt, model-invocation, and retrieval evidence               |
| <code>tool_action_authorized.v1</code>           | Tool-call evidence and policy evidence                         |
| <code>model_invocation_reconstructable.v1</code> | Invocation and runtime trace                                   |
| <code>human_approval_recorded.v1</code>          | Actor, role, scope, basis, and time                            |
| <code>incident_recovery_verified.v1</code>       | Failure and recovery remain distinguishable                    |
| <code>release_evidence_complete.v1</code>        | Release evidence completeness, not publication approval        |
| <code>ai_decision_with_limitations.v1</code>     | Evaluator/policy evidence with retained caveats                |
| <code>policy_reverification_required.v1</code>   | A stale, missing, or invalid trust condition blocks acceptance |

A profile evaluation returns its ID/version, passed and failed requirements, exact missing evidence, caveats, and a bounded result: `accepted`, `rejected`, `accepted_with_limitations`, or `requires_reverification`. The public Python surface and CLI can list built-in profiles. Profile outcomes can be persisted in the existing workspace evidence boundary; they do not create a second authority or automatically authorize a real-world action.

### 6.3 Human verification reports: legibility without false assurance

The `statewake.reports` package renders the established reliability verification-report contract as deterministic JSON and human-readable Markdown. Its candidate identity and digest bind a readable report to the reviewed object. Included, omitted, and missing evidence are kept separate, alongside source identities, artifact digests, residual risk, caveats, and the human decision still required. The gate vocabulary is deliberately non-binary: `PASS`,

FAIL, UNRUN-ENV (a prerequisite was unavailable), and UNKNOWN (the check is not sufficiently defined). Neither an unavailable check nor an unknown one becomes a pass. Redaction may conceal sensitive payload fields while retaining digest references.

The report's **verification decision** and **human approval status** are separate fields. Thus an acceptable evidence profile can remain *not approved* for publication, deployment, or a business action.

## 6.4 Durable backup, restore, and migration markers

Version 0.4.0 strengthens the v0.3.0 workspace rather than changing its three-role data model. The default local root is now `data/statewake/`; callers may choose an isolated root. Backup preserves the SQLite operational index, manifest, receipts, content-addressed artifacts, exports, and checksums as one local unit. Restore checks archive member paths, declared contents, digests, schema/format identity, and artifact bytes before reopening and verifying the workspace. An integrity sweep independently checks that stored artifact bytes still match their content-addressed paths.

Explicit migration markers distinguish initial workspace support, AI-contract compatibility, and profile-result compatibility. **The included SQLite physical schema and workspace manifest remain at version 1:** the latter two markers do not assert new physical tables or a second evidence store. Backup is distinct from the portable *dataset* ZIP, which remains distinct from a reliability *proof* bundle.

## 6.5 First-class producer integration, not vendor orchestration

Thin adapters normalize OpenTelemetry GenAI events; OpenAI Agents-style traces; LangChain callbacks; LlamaIndex retrieval/evaluator events; LangGraph runtime/checkpoint events; CI/CD records; and general evaluator results into typed AI contracts. Their `ContractCaptureResult` contains the normalized contract, an `EvidenceItem`, and explicit workspace-persistence helpers. Adapters accept mapping/object-shaped events and do not import optional vendor SDKs at the core package boundary; this avoids claiming tested native compatibility with every SDK version. Raw prompt, response, tool-input, and tool-output bodies are not stored by default. The host remains responsible for collecting the producer events and deciding what to admit.

## 6.6 Release trust: evidence completeness is not release permission

The `statewake.release_trust` family represents the **StateWake package's own release trust** separately from the reliability claims StateWake manages for other AI systems. A digest-bound bundle refers to package artifacts and sizes, source and dependency-lock identities, build provenance, tests, software bill of materials (SBOM), vulnerability scans, signature evidence or an explicit signing limitation, external provenance, and the human release-decision basis. Deterministic JSON round trips and tamper checks support independent inspection of that bounded record. The bundle can feed the existing `release_evidence_complete.v1` profile without replacing release-proof services or a human approval gate.

Absent signatures must be recorded as limitations rather than silently treated as successful signing. References to SBOMs, vulnerability scans, and provenance are not assertions that StateWake itself generated those artifacts. Likewise, a complete bundle supports a release decision; it does not by itself constitute publication authorization.

## 6.7 Implemented comparative validation: what the fixture harness establishes

The `statewake.validation_study` harness now provides executable, deterministic comparisons across **five representative workflows**: RAG answer, tool action, incident recovery, release verification, and human approval. For each workload it generates a clean and an injected-fault case under four *controlled representations*: final output only, conventional logs, structured traces, and the full StateWake evidence/profile representation. The shipped run comprises **40 cases (10 per representation)**, with **12 fault kinds in the catalog** and seven selected injected-fault occurrences in its fixed case matrix. Its report is digest-bound and renderable as JSON or Markdown.

| Controlled representation | Checkable properties / targeted | Fixture fault detections / injected | False positives / clean cases |
|---------------------------|---------------------------------|-------------------------------------|-------------------------------|
| Final output only         | 2 / 58                          | 0 / 7                               | 0 / 5                         |
| Conventional logs         | 2 / 58                          | 0 / 7                               | 0 / 5                         |
| Structured traces         | 4 / 58                          | 0 / 7                               | 0 / 5                         |
| StateWake full            | 58 / 58                         | 7 / 7                               | 0 / 5                         |

**Table.** The v0.4.0 *declared-fixture* comparison. Coverage is checkable/target properties; detection is mapped injected faults. Values are from the packaged `phase7-d0a7f7cb840c` report and reproduce when the harness is run against the released v0.4.0 source artifact.

These values require a precise interpretation. The harness **declares** which properties each representation captures and **maps** fault kinds to detections in its fixture logic; they are not observed detection rates from independently deployed logging or tracing products. The StateWake profile evaluator is exercised in its cases, but the aggregate 7/7 figure is not an end-to-end adversarial or live-model study. No human reconstruction-time, third-party baseline equivalence, statistical advantage, latency, or storage-cost finding follows from this table. The study establishes a reproducible *architectural capability and regression baseline* against which those harder experiments can be designed.

## 7. Integration and deployment surface

StateWake is a Python package (`statewake-ai`, imported as `statewake`) with a framework-neutral SDK, public Python API, CLI, and a WSGI verification boundary. The wheel metadata for v0.4.0 declares Python 3.11–3.13, core dependencies including PyNaCl, OpenTelemetry API, and filelock, plus the optional workspace extra. The public API contract version remains 1 through v0.4.0, as stated in the shipped changelog.

A typical composition is:

```
host workflow -> evidence adapter -> canonical receipt/store
                                   \-> workspace index / query
canonical evidence -> verification -> reliability state -> proof/decision
workspace query -> disclosure-filtered projection -> export / analysis
```

An adopter decides the producer identities, evidence admission policy, business thresholds, trust anchors, data sensitivity, deployment authentication, persistence placement, and operational review. The WSGI wrapper's hooks are integration surfaces, not a default multi-tenant service. Analytics operate on selected, disclosed projections rather than replacing authoritative evidence or changing the verified reliability decision.

## 8. Worked reliability lifecycle (illustrative)

Suppose an agent submits a compliance recommendation based on an external document, a tool result, and a rule evaluation. The following example illustrates how the implemented *types of record* can be composed; it is not a reproduced v0.3.0 benchmark or claim of live compliance integration.

**Initial occurrence.** The file adapter admits the document; the tool and evaluation adapters capture their own evidence with producer identities and digests. Provenance links the decision inputs, and verification checks the declared identities and edges. An accepted claim can be attested only when the chain's verification, reconciliation, and reliability conditions permit it.

**Discrepancy.** A document revision yields a different artifact digest. The earlier claim remains attached to the old evidence; a comparison and resulting state transition express the new disagreement. An operator can determine the *previously accepted basis*, rather than confusing it with the latest content.

**Recovery.** A corrected tool occurrence and a bounded reconciliation outcome become new evidence. If the declared conditions are met, the resulting reliability state can be *recovered*; the original failure and its evidence remain historically distinguishable.

**Operational use.** The v0.2.0 assurance function may require reverification, quarantine, or allow-with-limitations according to verified inputs and policy. The v0.3.0/v0.4.0 workspace can query historical occurrences by run/source identity and produce disclosure-bounded CSV, JSON, Parquet, XLSX, or a portable dataset bundle. A separate proof bundle—not the dataset ZIP—supports the applicable reliability-proof verification path. The v0.4.0 AI contracts can preserve the prompt, invocation, retrieval, tool, policy, and approval records relevant to the same occurrence; a selected claim profile states which are mandatory. A human verification report can disclose missing, unavailable, or unknown gates without converting the workflow decision into human approval.

## 9. Implementation and validation evidence

This paper is grounded in the **released StateWake v0.4.0 source archive**, including `src/statewake`, tests, architecture and release documentation, the versioned changelog, and the packaged Phase 7 study report. Unlike the earlier v0.3.0 distribution-only paper, this released artifact exposes the implementations and targeted tests for the new AI-contract, profile, reporting, workspace, integration, release-trust, and validation-study surfaces. The released package baseline is v0.4.0, and the public API contract remains version 1.

I reran the packaged deterministic comparative-study function from the supplied source. It reproduced study ID `phase7-d0a7f7cb840c`, **40 cases, five workloads, 12 catalogued fault kinds**, and the four reported fixture metrics in Table 3. A targeted regression selection covering AI contracts, validation-study reporting/workspace, human verification reports, producer integrations, release-trust bundles, workspace backup/restore, and AI-contract persistence completed with **60 tests passed** in the available environment. This is a bounded selection, **not** an assertion that the entire repository suite was rerun for this paper.

Earlier v0.1.0 deterministic scenarios and the B1–B4 *structural answerability* exercise remain historical design work. Their percentages must not be confused with the v0.4.0 fixture study or presented as measured benefits against products or people. Future comparative research requires matched live or realistically replayed executions, independently implemented baselines with equivalent access to evidence, injected faults, blinded human scoring, and measurement of storage, runtime, integration, and review costs.

## 10. Relationship to established work

StateWake does not invent provenance. W3C PROV supplies provenance concepts and validation constraints [1], [2]; OpenLineage concerns data pipeline entities and events [6]. It does not invent attestations or supply-chain provenance, which are addressed by in-toto and SLSA [3], [4]. It does not replace tracing/evaluation, represented by OpenTelemetry and LangSmith [5], [13], or durable workflow execution, represented by Microsoft Durable Task-style systems [14]. The agent-provenance literature already addresses fine-grained agent interactions and execution provenance [11], [12], while structured reasoning provenance explores a different, complementary record of agent decisions [15].

The distinctive **architectural proposition being tested** is the coupling of these existing kinds of evidence to explicit, persistent **reliability-state transitions** and their recovery, assurance, and data-lifecycle context. The v0.3.0 addition matters: a reliability claim can now be treated not only as a portable proof over one outcome but also as a durable, queryable history with controlled analytical projections. Version 0.4.0 adds typed AI-origin evidence, named minimum-evidence profiles, human-legible verification reports, producer-side adapters, local backup/restore, a separate release-trust record, and a deterministic study instrument. None of these claims requires that the upstream provider adopt StateWake as an agent runtime. This is a statement about StateWake's design center, **not** a claim that no other implementation can compose similar capabilities or that StateWake has been shown superior.

NIST AI RMF and cybersecurity guidance provide useful governance and security context [7]–[10]; citation does not mean StateWake is certified against those frameworks or provides their full organizational controls.

## 11. Limitations and research agenda

**Source correctness.** Valid digests and coherent provenance cannot establish the truth of the original source or the adequacy of a business policy. Host systems must define semantic evaluation and acceptance criteria.

**Trust and deployment.** Stolen signing keys, compromised producers, missing independent trust anchors, insecure hosts, cross-tenant leakage, and adversarial denial of service remain material risks. Included domain controls require correct integration and operations.

---

**Data completeness.** StateWake can verify the evidence admitted to it; it cannot automatically know about all omitted upstream events. Retention and disclosure decisions must balance reproducibility with confidentiality and deletion obligations.

**Scalability and portability.** Local SQLite reference persistence, optional analytical adapters, file locking, and schema-version checks provide an implemented foundation; comparative throughput, multi-writer scale, heterogeneous backend equivalence, and external verifier interoperability require workload-specific evidence. The SQLAlchemy surface does not by itself prove production equivalence across arbitrary databases.

**Comparative value.** The v0.4.0 fixture study assigns captured properties and fault-detection capability by representation, rather than empirically evaluating independent external products. It supplies a reproducible capability matrix, not a measured general advantage. The central research questions remain whether explicit reliability-state history improves reconstruction accuracy, failure/recovery attribution, independent verification, and operator effort—and at what storage and latency cost. No statistically supported effectiveness advantage is asserted here.

## 12. Conclusion

I built StateWake to make one proposition operational: a stateful AI outcome should be accompanied by a verifiable account of **what supports its current reliability state and how that state changed**. The v0.1.x core establishes evidence admission, provenance, integrity, state, recovery, and proof relationships. Version 0.2.0 extends that boundary with explicit security-assurance, trust, containment, lifecycle, incident, and decision models. Version 0.3.0 makes the resulting evidence operationally durable through a workspace index, historical query, integrity verification, retention, and structured exports. Version 0.4.0 supplies typed AI evidence, versioned claim profiles, reports suitable for human review, backup/restore, thin producer integrations, release-trust evidence, and an executable fixture comparison. The architecture does not replace upstream intelligence, existing standards, or deployment security. Its value is a narrower engineering contract: **claims remain tied to evidence and history rather than being reduced to a final success indicator**.

---

## References

### Standards and specifications

- [1] W3C, *PROV-DM: The PROV Data Model*, W3C Recommendation, 2013. <https://www.w3.org/TR/prov-dm/>
- [2] W3C, *Constraints of the PROV Data Model*, W3C Recommendation, 2013. <https://www.w3.org/TR/prov-constraints/>
- [3] in-toto, *in-toto Specifications and Attestation Framework*, stable v1.0. <https://in-toto.io/docs/specs/>
- [4] SLSA, *SLSA Specification v1.2: Provenance*, approved specification. <https://slsa.dev/spec/v1.2/provenance>
- [5] OpenTelemetry, *Observability Primer*, technical documentation. <https://opentelemetry.io/docs/concepts/observability-primer/>
- [6] OpenLineage, *Object Model*, technical specification/documentation. <https://openlineage.io/docs/spec/object-model/>
- [7] NIST, *AI Risk Management Framework (AI RMF 1.0)*, NIST AI 100-1, 2023. <https://doi.org/10.6028/NIST.AI.100-1>
- [8] NIST, *Cybersecurity Framework 2.0*, 2024. <https://www.nist.gov/cyberframework>
- [9] NIST, *Zero Trust Architecture*, SP 800-207, 2020. <https://doi.org/10.6028/NIST.SP.800-207>
- [10] NIST, *Secure Software Development Framework*, SP 800-218, 2022. <https://doi.org/10.6028/NIST.SP.800-218>

### Research and adjacent technical systems

- [11] R. Souza et al., *PROV-AGENT: Unified Provenance for Tracking AI Agent Interactions in Agentic Workflows*, arXiv:2508.02866, 2025 (preprint). <https://arxiv.org/abs/2508.02866>
- [12] Y. Wang et al., *From Agent Traces to Trust: Evidence Tracing and Execution Provenance in LLM Agents*, arXiv:2606.04990, 2026 (preprint). <https://arxiv.org/abs/2606.04990>
- [13] LangChain, *LangSmith Evaluation Types*, technical documentation. <https://docs.langchain.com/langsmith/evaluation-types>
- [14] Microsoft, *Durable Task for AI Agents*, technical documentation. <https://learn.microsoft.com/en-us/azure/durable-task/sdks/durable-task-for-ai-agents>
- [15] N. Vispute, *Reasoning Provenance for Autonomous AI Agents: Structured Behavioral Analytics Beyond State Checkpoints and Execution Traces*, arXiv:2603.21692, 2026 (preprint). <https://arxiv.org/abs/2603.21692>